

9주차 모델을 어떻게 만들었는지 어떻게 증명하는가

9주차 · 실험과 모델 관리

- ☐ 8장까지 만든 것은 데이터와 피쳐였음
- ☐ 이 장에서 처음으로 **모델 훈련**이 등장하고, 훈련이 등장하는 순간 새 질문이 따라옴
 - 그 모델을 어느 데이터로, 어느 설정으로, 어느 코드로 만들었는가
- ☐ 이 장은 그 답을 담당자의 기억이 아니라 기록 시스템에 두는 방법을 다룬다
- ☐ 이 장의 수치는 8장 피처에서 만든 훈련 쌍 6개로 후보 두 개를 실제로 훈련하고 등록한 결과다(MLflow 3.14.0, scikit-learn 1.9.0)

1. 오늘의 질문

- ☐ 석 달 전에 배포한 모델을 지금 다시 만들 수 있는가. 무엇을 알아야 다시 만들 수 있는가.
- ☐ 훈련 한 번에 무엇을 남겨야 "어떻게 만들었는가"에 답이 되는가.
- ☐ 성능 기준을 통과한 새 모델을 운영에 넣지 않는 것이 옳은 경우가 있는가.
- ☐ "지금 운영 중인 모델이 무엇인가"에 사람이 아니라 시스템이 답하게 하려면 무엇이 필요한가.

2. 시식 기록과 메뉴판으로 보는 이 장

- ☐ 새 메뉴를 개발하는 식당을 생각한다
 - 주방에서 하루에도 여러 번 시식을 하지만, 그중 손님에게 나가는 것은 메뉴판에 오른 것뿐임
 - 이 장의 네 개념이 그 구조에 하나씩 대응함

식당	이 장
시식 한 번의 기록지 — 재료 배합과 불 세기, 맛 평가 점수, 남겨 둔 사진과 레시피 사본	run 하나 — 파라미터, 메트릭, 아티팩트(artifact)
그날 쓴 재료의 납품 로트 번호	훈련 데이터의 지문(fingerprint) — sha256 해시
메뉴판에 이름과 번호로 올린 정식 메뉴	모델 레지스트리(registry)의 이름과 버전
"오늘의 추천" 팻말 — 손님은 팻말만 보고 주문한다	champion 별칭(alias) — 서빙이 이것만 바라본다
메뉴판에 올릴지 정하는 심사 기준과 심사자 서명	승격 게이트와 승격 검수 기록

- ☐ 대응의 핵심은 세 번째와 네 번째임
 - **시식 기록지를 아무리 많이 쌓아도 그것이 곧 메뉴판은 아니다**
 - 실험 기록과 배포 승인은 층이 다름
 - 손님이 팻말만 보듯 서빙 코드도 버전 번호가 아니라 별칭만 봄
- ☐ 비유가 깨지는 지점이 하나 있음

- 같은 레시피로 두 번 만든 요리는 맛이 조금씩 다르지만, 모델 훈련은 코드·데이터·파라미터·환경이 같으면 **결과가 완전히 같아야 한다**
- 이 장은 그 동일성을 수치로 확인함

3. run에 무엇을 남기는가 — 파라미터, 메트릭, 아티팩트

- 기록의 단위는 **run**이다
 - 훈련을 한 번 돌리면 run 하나가 생김
 - 실험(experiment)은 그 run들을 묶는 폴더다
 - run 하나에 들어가는 것은 세 종류로 나뉨
- **파라미터(params)**는 훈련의 입력 설정이다
 - 모델 종류, 하이퍼파라미터(hyperparameter), 그리고 이 실습에서는 훈련 데이터의 지문과 표본 수까지 넣음
 - "무엇을 넣었는가"에 답하는 항목임
- **메트릭(metrics)**은 훈련의 결과 수치다
 - 이 실습에서는 훈련 MAE(Mean Absolute Error, 평균 절대 오차) 하나임
- **아티팩트(artifacts)**는 결과 파일이다
 - 모델 파일, 훈련 데이터 사본처럼 수치로 줄일 수 없는 실물이 여기에 들어감
- 셋을 구분하는 이유는 찾는 방식이 다르기 때문이다
 - "학습률 0.01로 돌린 run 전부"는 파라미터 검색임
 - "MAE가 가장 작은 run"은 메트릭 정렬임
 - "그 run의 훈련 데이터를 열어 보자"는 아티팩트 내려받기임
 - 셋을 한 run에 함께 두어야 기록 단위가 완결됨

데이터 지문 — 도구가 지켜 주지 않는 한 가지

- 네 가지 입력 중 코드는 git이 지키고 파라미터는 도구가 자동으로 남기지만, "어느 데이터였는가"는 아무도 대신 지켜 주지 않는다
 - 훈련 데이터는 파일 경로가 같아도 내용이 바뀔 수 있기 때문임(7장의 정정 재실행을 떠올린다)
 - 그래서 훈련 데이터의 정규화 CSV에 sha256을 걸어 얻은 지문 96f6b6b3ce9dcdd1...을 모든 run의 파라미터에 넣었음
- 실습은 같은 훈련 데이터를 **아티팩트(CSV 사본)로도, 파라미터(지문)로도** 남김
 - 중복이 아니라 용도 분리임
 - 지문은 가볍고 비교가 되므로 "같은 데이터였나"에 즉답함
 - 사본은 무겁지만 실물이라 "그 데이터를 다시 보자"에 답함
 - 이 지문은 5장의 이벤트 지문, 7장의 산출물 해시와 같은 기법을 훈련 데이터에 다시 쓴 것임

후보 두 개의 실측 기록

- 훈련 데이터는 8장 피쳐 원천 9행(3지역 × 3일)에서 만든 (전일 건수 → 당일 건수) 쌍 6개임
 - 강남(11680)이 8→6과 6→9, 마포(11440)가 5→5와 5→6, 관악(11620)이 5→5와 5→5로 각 지역에서 2쌍씩 나옴
- 여기에 후보 두 개를 훈련했음
 - 하나는 전체 평균을 그대로 내놓는 베이스라인(baseline, 기준선 모델 — DummyRegressor, strategy=mean)임
 - 다른 하나는 전일 건수의 1차 함수인 선형회귀(LinearRegression)임
- 기록된 메트릭은 baseline_mean이 **1.0000**, linear가 **1.0455**임
- 선형회귀의 학습된 계수는 slope **0.4091**, intercept **3.6818**임
- 두 run 모두 파라미터에 n_samples=6과 data_sha256_16=96f6b6b3ce9dcdd1이 들어 있음
- 데이터가 6쌍뿐인 이유가 여기서 드러남
 - 이 값들은 전부 손으로 계산됨
 - 베이스라인의 예측값은 y 평균 $(6+9+5+6+5+5)/6 = 36/6 = 6.0$ 임
 - 잔차 절댓값 0, 3, 1, 0, 1, 1의 합 6을 6으로 나누면 MAE **1.0**이다
 - 선형회귀의 계수는 최소제곱 공식으로 $\text{slope} = 9/22 = 0.4091$, $\text{intercept} = 81/22 = 3.6818$ 이 되어 실측과 정확히 일치함
- 주목할 결과가 하나 있음
 - **더 복잡한 모델이 기준선에 미달했다**(1.0455 > 1.0000)
 - 6쌍으로는 의미 있는 기울기를 배울 수 없으므로 당연한 결과임
 - 이 당연함이 5절의 승격 판정을 사실대로 보여 줌
- 여기의 MAE는 **훈련 데이터 위의 훈련 오차**라는 단서를 붙여 둬
 - 6쌍을 훈련과 평가로 쪼개는 것은 의미가 없어 나누지 않았음
 - 실무라면 승격 판정의 근거는 훈련 오차가 아니라 보류(holdout) 데이터의 평가 지표여야 한다
 - 이 실습의 관찰 대상은 모델 품질이 아니라 기록과 등록의 구조다

4. 훈련도 함수다 — 같은 설정이면 같은 모델

- 7장은 배치를 입력의 결정적 함수로 만들었음
 - 같은 입력 파일이면 같은 산출물이 나오고 해시까지 같았음
- 훈련도 같은 관점으로 봄
 - **모델 = f(코드, 데이터, 파라미터, 환경)** 이다
 - 네 입력이 전부 같으면 같은 모델이 나와야 한다
- 다만 훈련에는 배치에 없던 입력이 하나 숨어 있음
 - 초기화·셔플·샘플링이 난수를 쓰는 알고리즘이라면 seed까지 입력에 넣어 고정해야 함수가 된다
 - 이 실습의 두 후보는 결정적 알고리즘이라 seed 없이도 결정적임
- **아무것도 기록하지 않으면 재현이 아니라 새 모델이 나온다**
 - 석 달 전 모델의 성능이 떨어져 "그때 모델을 다시 만들어 비교하자"고 할 때, 그때의 데이터·피쳐·파라미터·코드 중 하나라도 확실하지 않으면 다시 만든 것은 그 모델이 아님

- 비교 실험 자체가 성립하지 않음
- 이것이 게으름의 결과가 아니라는 점이 중요함
 - 실험은 하루에 파라미터를 다섯 번 바꾸고 데이터를 두 번 갈아 끼우는 것이 정상임
 - 그 속도에서 "나중에 정리하자"는 반드시 실패함
 - 그래서 기록은 실험 **후**의 문서화가 아니라 실험 **중**의 자동 기록이어야 한다

재현성 실측

- 선형회귀 후보를 같은 데이터(지문 동일)·같은 파라미터로 한 번 더 훈련했음
 - train_mae는 1.0455에서 다시 1.0455가 나왔음(mae_equal=true)
 - 계수도 slope 0.4091·intercept 3.6818로 같았음(coef_equal=true)
 - 7장의 "재실행 산출물 해시 동일"이 여기서 "재훈련 모델의 메트릭·계수 동일"이 된 것이며, 같은 원리의 모델판임
- 거꾸로 읽으면 이 확인은 **회귀 검사**가 된다
 - 아무것도 바꾸지 않았는데 메트릭이 달라졌다면 네 입력 중 무언가가 몰래 바뀐 것임
 - 이때 지문 파라미터가 어느 쪽인지 즉시 갈라 줌
 - 지문이 같은데 결과가 다르면 코드·환경·난수를 의심함
 - 지문이 다르면 데이터를 의심함

재현 실패의 원인을 분리하는 순서

- "지난달과 같은 설정으로 돌렸는데 MAE가 다르다"는 신고가 들어왔다고 하자
 - 기록이 있으면 이것은 추리가 아니라 대조 절차가 됨
 - 값싼 대조를 먼저, 비싼 대조를 나중에 함
- 1. **파라미터 대조** — 두 run의 파라미터를 필드별로 비교한다.
 - 다르면 종결임
 - "같은 설정"은 기억이고 파라미터 기록은 사실이다
- 2. **데이터 지문 대조** — 파라미터가 전부 같아도 data_sha256_16이 다르면 데이터가 바뀐 것이다.
 - 경로가 같아도 지문이 다르면 내용이 다르다
- 3. **환경 대조** — 앞의 둘이 같다면 라이브러리 버전을 본다.
 - 수치 라이브러리의 마이너 버전 차이가 부동소수점 수준의 차이를 만드는 일은 실재한다
- 4. **난수 확인** — 셋 다 같은데 다르면 seed가 고정되지 않은 난수를 찾는다.
- 이 절차 전체가 성립하려면 1~2단계의 기록이 **훈련 시점에** 이미 남아 있어야 한다
 - 실패한 뒤에는 만들 수 없는 기록임
 - 정보공개 청구나 감사에서 "이 모델의 판단 근거를 설명하라"는 요구가 왔을 때, 모델 파일만 가진 기관이 답하지 못하는 이유도 같음
 - 재현성은 연구자의 결벽이 아니라 **책임 소재의 전제 조건**이다

5. 레지스트리와 승격 게이트 — 통과와 교체는 다른 판단

- ☐ 실험을 계속하면 run은 수십, 수백 개가 쌓임
 - 그것이 정상이지만, 서버 시스템에 "run 목록에서 알아서 좋은 것을 골라 쓰라"고 할 수는 없음
- ☐ 배포는 다른 규율을 요구함
 - 후보는 **이름 있는 모델의 번호 있는 버전**이어야 함
 - 지금 운영 중인 것이 어느 버전인지 한 곳에서 답해져야 함
- ☐ 그 관문이 **모델 레지스트리**다
 - run이 개별 실행의 기록이라면, 레지스트리는 하나의 이름 아래 배포 후보를 버전으로 관리하는 별도 저장소다
 - 같은 이름 아래 v1, v2, v3가 쌓이고 각 버전은 자기를 만든 run을 가리키므로 기록과의 연결이 끊기지 않음
- ☐ 운영 중인 버전을 가리키는 방법은 **별칭(alias)** 이다
 - 버전에 champion 같은 가변 이름표를 붙이고, 소비자는 버전 번호가 아니라 `models:/complaint_daily_forecaster@champion`으로 모델을 불러옴
 - 별칭을 v1에서 v2로 옮기는 것이 곧 운영 모델 교체이며, 소비자 코드는 한 줄도 바뀌지 않음
 - 8장에서 서버가 피처를 정의 이름으로 조회했듯, 여기서는 모델을 역할 이름으로 조회함

승격 게이트 실측

- ☐ 등록 모델 이름은 `complaint_daily_forecaster`임
- ☐ 승격 규칙은 두 조건이다
 - 절대 게이트는 $\text{train_mae} \leq 1.05$ 임
 - 상대 조건은 기존 champion보다 개선일 것임
- ☐ 두 후보를 차례로 등록한 판정은 다음과 같다

버전	후보	train_mae	게이트(≤ 1.05)	champion 대비	판정
v1	baseline_mean	1.0000	통과	최초 — 비교 대상 없음	champion 별칭 부여(승격)
v2	linear	1.0455	통과	$1.0455 > 1.0000$, 개선 없음	승격 반려 — v2 등록은 유지

표 9.1 레지스트리 실측 — 등록·게이트·별칭(ch9_promotion_record.json)

- ☐ v2가 반려된 이유를 정확히 읽어야 한다
 - 절대 게이트는 통과했음($1.0455 \leq 1.05$)
 - 탈락 사유는 상대 조건임
 - 지금 운영 중인 것보다 나아야 교체할 이유가 있으므로, **절대 기준 통과와 교체 가치는 다른 판단**이고 승격 규칙이 둘 다 명시해야 하는 이유가 여기에 있다
- ☐ 한쪽만 두면 각각 다른 사고가 난다

- 절대 게이트만 두고 상대 조건을 빼면 기준을 겨우 넘긴 후보가 더 나은 현행 모델을 밀어내고 운영에 들어감
- 반대로 상대 조건만 두고 절대 게이트를 빼면 현행보다 조금 낮기만 하면 승격되므로, 두 모델이 모두 업무에 못 쓸 수준이어도 교체가 계속 일어남
- 반려된 v2는 삭제되지 않고 **버전으로 남는다**
 - 이것은 낭비가 아니라 감사 기록임
 - "선형회귀를 시도했고 이 수치로 반려되었다"가 레지스트리에 있으면 다음 담당자가 같은 시도를 반복하기 전에 판정 이력을 확인함
 - 별칭 이동이 승격이라면 같은 동작의 역방향이 **롤백(rollback)** 이므로, 버전을 지우지 않는 것이 곧 복귀 대상을 남겨 두는 일이기도 함
- 승격 후의 소비도 실측했음
 - champion 별칭으로 로드하면 v1이 오고, 8장 온라인 스토어의 강남구 최신 피쳐인 전일 건수 9.0을 넣으면 예측은 **6.00**임
 - champion이 평균 예측기이므로 입력과 무관하게 훈련 y 평균 6.0을 돌려줌
 - 입력 9가 예측에 반영되지 않는 것은 버그가 아니라 v1의 정의다

등록의 검증은 로드까지다

- 이 실습을 만들며 실제로 겪은 오류를 남겨 둬
 - 버전 등록의 경로를 `runs:<run_id>/model`이라는 관례 문자열로 조립해 넘겼더니 등록까지는 아무 오류도 없었지만, 별칭으로 로드하는 순간 `Could not find an 'MLmodel' configuration file` 예외가 났음
 - 수정은 `log_model`의 반환값(`model_info.model_uri`)을 등록에 넘기는 것이었음
- 요점은 특정 버전의 내부 구조가 아니라 실패의 형태다
 - **쓰기는 성공 로그를 남겼고 파손은 읽기에서야 드러났다**
 - 8장의 빈 온라인 스토어와 같은 유형임
 - 그래서 등록 성공의 정의를 별칭 로드와 예측 계산까지로 넓힘

6. 승격 검수 기록 — 누가 승인했는가

- 앞의 세 절이 남긴 기록은 "어떻게 만들었고 얼마나 좋은가"에 답함
- 공공 감사는 거기에 더해 묻는다
 - 누가 만들었고, 무슨 목적으로, 어느 기간의 어떤 출처 데이터로, 어떤 제한 아래 쓰이는가
- 이 항목들은 성능과 무관하지만 책임과 직결되고, 나중에 소급 작성이 사실상 불가능함
 - 담당자는 이동하고 기억은 사라짐
 - 그래서 훈련 시점에 run의 태그(tag)로 함께 남김
 - 실습이 모든 run에 붙인 최소 집합은 소유자, 목적, 데이터 출처, 데이터 기간 넷임
- 승격 판정 자체도 문서로 남는다
 - 공공 운영에서 승격은 기술 행위가 아니라 **승인 행위**이므로, 누가 어떤 규칙으로 어떤 근거를 보고 통과시켰는지가 남아야 함

- 실습은 판정 직후 그 내용을 ch9_promotion_record.json으로 자동 생성함
- 사람이 따로 쓰는 문서가 아니라 판정 로직의 실측 출력에서 뽑아낸 것임
- 이 기록에 담긴 것은 여섯 가지다
 - 모델 이름과 현재 champion(complaint_daily_forecaster / v1)
 - 승격 규칙($\text{train_mae} \leq 1.05$ 그리고 기존 champion 대비 개선)
 - 판정 이력(v1 승격 1.0000, v2 반려 1.0455)
 - 재현성 확인($\text{mae_equal}=\text{true}$, $\text{coef_equal}=\text{true}$)
 - 데이터 지문(96f6b6b3ce9dcdd1...)
 - 검수자(검수 담당 — 가상 표기)
 - 여기에 제한 사항으로 "훈련 데이터 6쌍(교육용 극소 데이터) — 성능 수치는 메커니즘 관찰용"이 붙음
- 재현성 확인이 별도 항목으로 들어간 이유를 짚어 둬
 - 자동 메트릭 게이트는 표 9.1의 두 조건이고, 재현성은 그와 별개인 승인 검수 요건임
 - **재현되지 않는 run은 게이트 수치가 좋아도 승격 대상이 아니다**
 - 그 수치가 코드와 데이터의 성질인지 우연의 산물인지 구분할 수 없기 때문임
- 검수 기록에서 가장 자주 비는 칸은 검수자다
 - 자동 게이트가 판정을 내리면 "시스템이 통과시켰다"로 끝내기 쉽지만, 행정 처분이나 자원 배분에 쓰이는 모델이라면 그 판정을 확인한 사람이 남아야 한다
- 설계할 때 정할 것이 셋이다
 - 게이트를 통과했을 때 승격을 자동으로 둘 것인가 사람 확인을 요구할 것인가
 - 게이트 미달인데 승격해야 하는 상황에서 누가 예외를 승인하는가
 - champion 별칭을 옮길 권한을 누가 갖는가
 - 셋째가 특히 실무적임
 - 모델을 훈련하는 사람과 별칭을 옮기는 사람이 같으면 게이트는 자기 검사에 그친다

7. 실습 9.1 승격 게이트 설계와 MLflow 실험 기록·모델 등록

실습 목표

- 승격 게이트 규칙을 실행 **전에** 스스로 정하고 근거를 적는다
- 그다음 8장 피쳐로 후보 둘을 훈련해 run으로 기록하고, 재현성을 확인한 뒤 등록 → 게이트 판정 → champion 별칭 → 별칭 로드·예측까지 실행한다
- 마지막으로 자기 게이트를 실측 메트릭에 적용해 스크립트의 판정과 대조한다
- 순서에 의도가 있음
 - MLflow를 어떻게 호출하는지에는 고를 것이 없지만, "무엇이 얼마나 좋아져야 운영 모델을 바꾸는가"는 전부 선택임
 - 그 선택을 실행 결과를 본 뒤에 하면 규칙이 아니라 사후 합리화가 됨

1단계 설계 — 승격 게이트 설계표

□ 다음 요구사항을 읽고 아래 표의 빈칸을 채운다

- 표 9.1의 규칙($\text{train_mae} \leq 1.05$, champion 대비 개선)은 완성된 예시 답안이므로 그대로 옮겨 적지 않는다

□ 요구사항은 다섯 줄이다

- 지역구별 익일 민원 건수를 예측해 다음 날 상담 인력 배치에 씀
 - 예측이 낮으면 대기 시간이 늘고 높으면 인력이 논다
- 재훈련은 월 1회이고, 모델을 교체하면 상담 인력 운영 계획도 함께 바뀜
- 감사는 운영 모델이 바뀔 때마다 "왜 바꿨는가"에 답할 것을 요구함
- 훈련 데이터의 원천은 7장 확정 집계이며 정정 재실행으로 과거 값이 바뀔 수 있음
- 담당자는 2년 주기로 바뀜

항목	내가 정한 값	근거(요구사항의 어느 문장에서 나왔는가)
승격 판정 지표		
절대 게이트 임계값		
상대 조건(champion 대비 개선 폭)		
재현성·데이터 지문 요건		
운영 태그 최소 집합		
승인 주체와 예외 승인 경로		
이 게이트가 깨지는 조건		

표 9.2 승격 게이트 설계표 — 학습자 작성용

□ 지킬 것이 넷이다

- 첫째, 근거 칸에 "기본값이라서"를 쓰지 않는다
- 둘째, 임계값은 요구사항 문장에서 끌어온다
 - 지금 사람이 배치를 정하는 방식의 오차, 또는 오차가 얼마를 넘으면 인력 계획이 틀어지는지 중 하나에 닿아야 함
- 셋째, 상대 조건에는 **얼마나** 좋아져야 하는지를 적는다
 - "개선되면"과 "0.1 이상 개선되면"은 다른 규칙임
- 넷째, "깨지는 조건"은 양방향으로 검토한다
 - 느슨해서 나쁜 후보를 통과시키는 쪽과, 빡빡해서 어떤 후보도 통과하지 못해 오래된 모델이 계속 남는 쪽이 둘 다 실패임

□ 표를 채운 뒤, 실행하기 전에 한 줄을 더 적는다

- 자기 게이트로 두 후보의 판정이 각각 어떻게 될 것 같은지, 그리고 실습이 끝났을 때 champion이 몇 번 버전일 것 같은지다

2단계 실행

- ☐ Python 3.10 이상이면 되고 서버 상주가 없음
- ☐ 추적과 레지스트리 백엔드는 로컬 SQLite(sqlite:///data/output/mlflow.db)임
 - 레지스트리는 파일 백엔드로 동작하지 않고 DB 백엔드를 요구함

```
cd practice/chapter9
python3 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt
python run_chapter9.py
```

- ☐ 기록은 run 컨텍스트 안에서 세 종류를 각자의 API로 남기는 것이 전부다

```
with mlflow.start_run(run_name="linear") as run:
    mlflow.log_params({"model_type": "LinearRegression", "data_sha256_16": "f0f:161"})
    mlflow.log_metric("train_mae", mae)
    mlflow.log_artifact(str(csv_path), artifact_path="training_data")
    model_info = mlflow.sklearn.log_model(model, name="model")
```

전체 코드는 practice/chapter9/code/9-1-mlflow-tracking.py 참고

실행 결과 확인 사항

- ☐ 스크립트는 네 시나리오를 차례로 돈다
 - 후보 둘의 훈련·기록(3절), 같은 설정 재실행(4절), 등록·게이트·별칭(5절), 별칭 로드·예측과 검수 기록 생성(6절)이다
 - 1. **훈련 데이터**: 쌍 6개, 지문 96f6b6b3ce9dcdd1...
 - 2. **시나리오 1**: baseline_mean train_mae=1.0000, linear train_mae=1.0455(slope 0.4091, intercept 3.6818)
 - 3. **시나리오 2**: 메트릭 동일=True, 계수 동일=True
 - 4. **시나리오 3**: v1 승격, v2 반려(게이트는 통과했으나 개선 없음), champion=v1
 - 5. **시나리오 4**: 입력(전일 건수) 9.0 → 예측 6.00
 - 6. **마지막 줄**: CH9_RUN_PASS
- ☐ 산출물은 네 가지다
 - mlflow.db가 추적·레지스트리 백엔드임
 - ch9_experiment_report.json이 이 장 인용 수치의 증거 파일임
 - ch9_promotion_record.json이 승격 검수 기록의 정본임
 - training_pairs_*.csv가 아티팩트로 기록된 훈련 데이터다

3단계 대조 — 내 게이트를 실측에 적용하기

- ☐ 실측은 baseline_mean 1.0000, linear 1.0455이고 스크립트 판정은 v1 승격·v2 반려다
- ☐ 여기에 자기 게이트를 다시 적용해 판정이 같은지 봄
- ☐ 절대 게이트만 걸었다면, 임계 1.05에서는 둘 다 통과하고 1.0에서는 linear만 탈락한다
 - 어느 쪽이든 절대 기준 하나로는 "교체할 이유"를 판정하지 못한다는 것이 v2에서 드러남

- 상대 조건에 개선 폭을 적었다면 이번 실행에서는 판정이 달라지지 않는다
 - linear는 개선이 아니라 악화($1.0455 > 1.0000$)이므로 어떤 폭을 요구하든 반려다
 - 폭 규정의 효과는 후보가 0.9990을 냈을 때 갈림
 - "개선되면 승격"은 교체하고 "0.05 이상 개선되면 승격"은 교체하지 않는다
- 예측을 "새 후보가 이긴다"로 적었다면 무엇을 빠뜨렸는지 확인함
 - 복잡한 모델이 단순한 기준선을 이긴다는 보장은 없음
 - 승격 게이트는 새 후보를 통과시키는 장치가 아니라, 교체할 이유가 없을 때 교체하지 않게 막는 장치다
- 판정 지표를 MAE가 아닌 것으로 골랐다면 이번 증거로는 검증되지 않음
 - 기록된 메트릭이 train_mae 하나이기 때문임
 - 이 경우 표 9.2의 "깨지는 조건" 칸에 "판정에 필요한 지표가 run에 기록되지 않으면 게이트가 작동하지 않는다"를 적는다
 - 게이트 설계와 기록 설계가 붙어 있다는 것이 여기서 드러남
- 마지막으로 표 9.2의 "이 게이트가 깨지는 조건" 칸을 다시 쓴다
 - 제출하는 설계표는 이 수정을 반영한 최종본이다

8. 핵심 요약

- 1. run 하나는 파라미터·메트릭·아티팩트 세 항목으로 완결된다
 - 입력 설정과 결과 수치와 결과 파일은 찾는 방식이 달라서 나누어 남김
 - 훈련 데이터는 지문(파라미터)과 사본(아티팩트) 양쪽으로 남기는데, 지문은 "같은 데이터였나"에 즉답하고 사본은 실물을 다시 보게 해 줌
- 2. 데이터의 동일성은 도구가 자동으로 보장하지 않는다
 - 코드는 git이, 파라미터는 도구가 지키지만 훈련 데이터는 경로가 같아도 내용이 바뀔
 - 그래서 sha256 지문 96f6b6b3ce9dcdd1...을 파라미터로 직접 기록했음
- 3. 같은 설정을 다시 돌리면 같은 모델이 나온다
 - 선형회귀 재실행의 train_mae는 1.0455로 같았고 계수도 slope 0.4091·intercept 3.6818로 같았음
 - 7장의 해시 동일이 여기서 메트릭·계수 동일이 되었음
 - 거꾸로 쓰면 몰래 바뀐 입력을 잡는 회귀 검사가 됨
- 4. 절대 기준 통과와 교체 가치는 다른 판단이다
 - v2는 게이트($1.0455 \leq 1.05$)를 통과했지만 champion인 v1(1.0000)보다 나쁘므로 반려되었음
 - 절대 게이트만 두면 겨우 넘긴 후보가 더 나은 현행을 밀어냄
 - 상대 조건만 두면 둘 다 못 쓸 수준이어도 교체가 계속됨
- 5. 승격은 기술 행위가 아니라 승인 행위다
 - 규칙·판정 이력·재현성 확인·데이터 지문·검수자를 담은 검수 기록이 판정 실물에서 자동 생성되어야 감사에 답할 수 있음
 - 훈련하는 사람과 champion 별칭을 옮기는 사람이 같으면 게이트는 자기 검사에 그침

9. 과제

□ 실습을 실행한 뒤 다음 세 가지를 LMS에 업로드한다

1. 표 9.2 승격 게이트 설계표의 최종본(3단계 대조를 반영한 것)
2. `practice/chapter9/data/output/ch9_experiment_report.json` 파일 그대로
3. 아래 두 물음에 대한 답(각각 세 문장 이내)

□ 첫째, 본인의 `ch9_experiment_report.json`에서 `registry.champion_version`과 두 버전의 `version.train_mae-metric_gate-metric_gate_pass-promoted`를 옮겨 적는다

- 그다음 절대 게이트를 통과했는데도 승격되지 않은 버전의 `train_mae`와 `champion`의 `train_mae`를 나란히 적고, 반려된 이유를 쓴다

□ 둘째, 절대 게이트만 두고 상대 조건을 두지 않으면 어떤 사고가 생기는지, 반대로 상대 조건만 두고 절대 게이트를 두지 않으면 어떤 사고가 생기는지 각각 한 문장으로 쓴다

□ 수업일 포함 7일 이내에 제출한다

10. 더 알아보기

□ 이 장에서 다루지 않은 내용은 실무교재 `lecture/docs/chapter9.md`에 있다

□ **단계(stage)와 별칭의 대응** — 개발·검증·운영·폐기 4단계를 별칭으로 옮기는 표:
`lecture/docs/chapter9.md §9.3`

□ **민간의 승격 지표와 상시 실험 문화** — CTR·CVR·ARPU 이중 게이트, A/B 테스트와 새도 배포의 차이: §9.3

□ **모델 카드와 데이터시트** — 이용자·외부에 모델을 설명하는 공개용 양식과 이 장 태그의 관계: §9.4

□ **실습과 운영의 간극** — 데이터 규모·평가 방식·추적 서버·승인 워크플로에서 축소된 지점 목록:
`lecture/docs/chapter9.md` 실습 9.1

□ **아티팩트 실물과 UI 탐색** — `mlruns/` 디렉터리 구조와 `mlflow ui` 사용: `practice/chapter9/` 및 §9.2